Лабораторна робота №8

Підготував: Коваль Олексаднр Дмитрович
Група: 2ПІ-25Б
Підгрупа: 1

Тема: викоистанная функцій для створення та читання документів, таблиць, операцій з ними
Мета: Опанувати введення/виведення даних у файли, робота з текстовими та бінарними файлами.

Блок-схема

1. Запит: "Відкрити файл flight.csv?"
   - Так → Відкрити файл
   - Ні → Вивести "Error opening file flight.csv" → Кінець
2. Зчитати заголовок
3. Для кожного рядка у файлі:
   - Якщо рядок порожній → Пропустити
   - Викликати `parseLine(line, flightNo, seg)`
     - Якщо успішно → Додати сегмент до flights[flightNo]
     - Інакше → Пропустити
4. Закрити файл
5. Для кожного рейсу у flights:

- Викликати `buildRoute(segs)`

- Якщо розмір маршруту ≤ 2 → Пропустити

- Знайти унікальні проміжні аеропорти (без першого і останнього)

- Для кожного проміжного аеропорту → збільшити лічильник у intermediateCount

6. Якщо intermediateCount порожній:

- Вивести "No intermediate airports found."

- Кінець

7. Знайти аеропорт з найбільшою кількістю рейсів (bestAirport)

- Вивести:

"Проміжний аеропорт з найбільшою кількістю рейсів: bestAirport (bestCount)"

Кінець

Код Програми

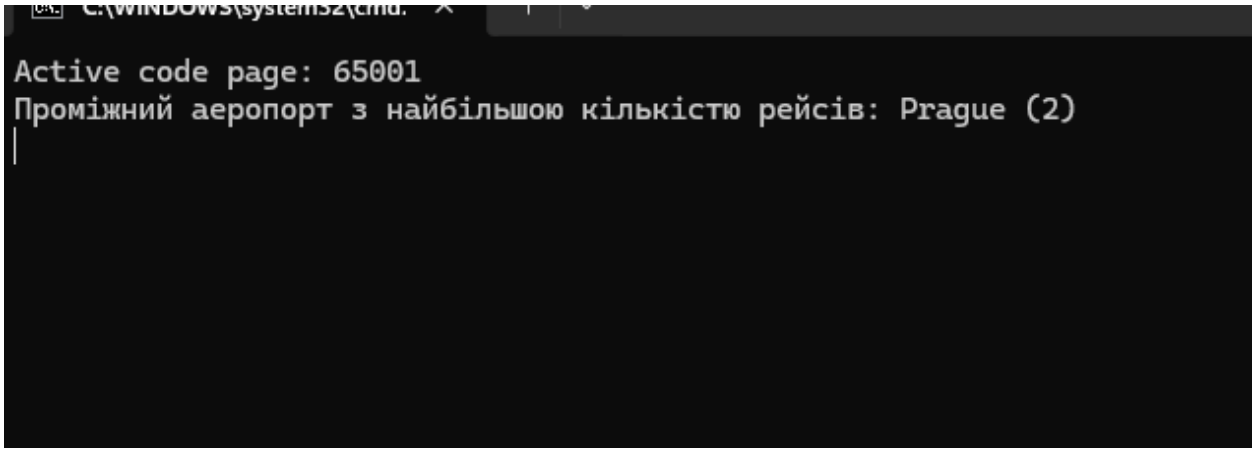
```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <algorithm>
```

```
using namespace std;

struct Segment { // good
    string depTime;
    string arrTime;
    string depAirport;
    string arrAirport;
};

bool parseLine(const string& line, string& flightNo, Segment& seg) { // +-
    size_t start = 0, pos = line.find(',', start);
    if (pos == string::npos) return false;
    flightNo = line.substr(start, pos - start);
    start = pos + 1;

    pos = line.find(',', start);
    if (pos == string::npos) return false;
    seg.depTime = line.substr(start, pos - start);
    start = pos + 1;

    pos = line.find(',', start);
    if (pos == string::npos) return false;
    seg.arrTime = line.substr(start, pos - start);
    start = pos + 1;

    pos = line.find(',', start);
    if (pos == string::npos) return false;
    seg.depAirport = line.substr(start, pos - start);
    start = pos + 1;

    seg.arrAirport = line.substr(start);
    if (seg.arrAirport.empty()) return false;

    return true;
}

vector<string> buildRoute(vector<Segment>& segments) {
    vector<string> route;
    if (segments.empty()) return route;
```

```
    sort(segments.begin(), segments.end(), [](const Segment& a, const
Segment& b) {
        return a.depTime < b.depTime;
    });
    route.push_back(segments[0].depAirport);
    for (const auto& seg : segments) {
        route.push_back(seg.arrAirport);
    }
    return route;
}

int main() {
        // cout << "Which file to open? \n";
        // cout << "Print your file name:\n";
        // string fileName;
        // cin >> fileName;
        // ifstream file("flight.csv");
        ifstream
file(R"(c:\Users\muaro\Documents\GitHub\Learning-assistent\labs\c\base_lab
_pract\number8-12-my\flight.csv)");
        if (!file.is_open()) {
            cerr << "Error opening file flight.csv" << endl;
            cin.get();
            cin.ignore(10000, '\n');
            return 33;
        }

    string header;
    getline(file, header);

    map<string, vector<Segment>> flights;
    string line;
    while (getline(file, line)) {
        if (line.empty()) continue;
        string flightNo;
        Segment seg;
        if (parseLine(line, flightNo, seg)) {
            flights[flightNo].push_back(seg);
        }
    }
```

```
    file.close();

    map<string, int> intermediateCount;
    for (const auto& pair : flights) {
        vector<Segment> segs = pair.second;
        vector<string> route = buildRoute(segs);
        if (route.size() <= 2) continue;

        set<string> uniqueMids;
        for (int i = 1; i < route.size() - 1; ++i) {
            uniqueMids.insert(route[i]);
        }

        for (const string& mid : uniqueMids) {
            intermediateCount[mid] += 1;

        }
    }

    if (intermediateCount.empty()) {
        cout << "No intermediate airports found." << endl;
        return 33;
    }

    string bestAirport;
    int bestCount = 0;
    for (const auto& pair : intermediateCount) {
        if (pair.second > bestCount) {
            bestCount = pair.second;
            bestAirport = pair.first;
        }
    }

    cout << "Проміжний аеропорт з найбільшою кількістю рейсів: "
         << bestAirport << " (" << bestCount << ")" << endl;
    cin.get();
    cin.ignore(10000, '\n');
    return 0;
}
```

Результат Програми



Висновок

На цій лабораторній роботі я навчився використовутивати функції для чатння та створення файлів з даними, операціями з ними та фільтрувати їх.

Відповіді на питання

10. Логічний файл — це об’єкт у програмі для роботи з даними;

Фізичний файл — це реальний файл на диску.

11. Введення з файлу:

Рядкові дані — через `getline(file, str)`;

Дійсні — через `file >> x;` або парсинг рядка.

12. Форматний вивід:

Символьний — `cout << ch;`

Рядковий — `cout << str;`

Дійсний — `cout << fixed << setprecision(2) << x;`